

Course code	Course Title	No. of Periods/Weeks	Total No. of periods	Marks for FA	Marks for SA
CM-106	Programming in C	5	150	20	80

Time Schedule

Chapter No.	Chapter/Unit Title	No. of Periods	Marks	No. of Short Answer Questions	No. of Essay Type Questions	CO's Mapped
1.	Introduction to C Language	20(10,10)	16	2	1	CO1,CO2
2.	Input and output statements, Operators and Expressions in C.	25 (8,12, 5)	16	2	1	CO1,CO2, C3
3.	Decision making, iterative and other control statements	40 (5,20,15)	26	2	2	CO1,CO2, CO3
4.	Arrays and strings, Structures and Unions	30 (5,15,10)	26	2	2	CO1,CO2 ,CO3
5.	User defined functions, pointers, file management and pre-processor directives.	35 (3,5,10,10,7)	26	2	2	CO1,CO2 ,CO3,CO4, CO5
Total Periods		150	110	10	8	

Course Objectives	- To Relate basics of programming language constructs using C Language - To classify and implement data types, derived data types, pointers, files, statements - To analyse and develop effective modular programming - To construct mathematical, logical and scientific problems and real time applications using C language
--------------------------	---

CO NO	COURSE OUTCOMES
CO1	CM-106.1 Develop, compile and debug programs using C- fundamentals and different programming statements in C language.
CO2	CM-106.2 Evaluate various operations using primary and derived data types in C.
CO3	CM-106.3 Analyze programs using predefined functions, modules and recursive

		techniques
CO4	CM-106.4	Write scientific and logical programs using pointers, file pointers
CO5	CM-106.5	Develop programs using information passing

CO-PO/PSO MATRIX

CO NO.	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PSO1	PSO2	PSO3
CM-106.1	3		2	2				3	2	
CM-106.2				2				3	3	
CM-106.3		2	3	3				3	1	2
CM-106.4	3		1	1	2			3	2	2
CM-106.5			2	2		2	2	3	2	3
Average	3	2	2	2	2	2	2	3	2	2.3

3=Strongly mapped , 2=moderately mapped, 1=slightly mapped

Learning Objectives

1.0 Introduction to C-Language

- 1.1** Describe the history of C-language, structure of C-language program
- 1.2** Describe the programming style of C language
- 1.3** Explain the steps involved in Editing, compiling ,executing and debugging of C program
- 1.4** Describe character set, C-Tokens, Keywords, Identifiers, Constants, Variables
- 1.5** Define Data Type
- 1.6** Classify Data Types and explain them with examples.
- 1.7** Explain declaration of constants and variables
- 1.8** Explain initializing values to variables in declaration
- 1.9** Explain about user defined data types with a simple program
- 1.10** Explain the usage of type qualifiers

2.0 Input and output statements, Operators and Expressions in C

- 2.1** Explain the importance of Pre-processor Directive #include
- 2.2** Illustrate
 - 2.2.1** Reading a character using getch(), getche() and getchar()
 - 2.2.2** writing a character using putch(), putchar()
 - 2.2.3** formatted input using scanf() & write sample programs using it.
 - 2.2.4** formatted output using printf() & write sample programs using it.
- 2.3** Explain character functions
- 2.4** Define an operator, an expression

- 2.5** Explain
 - 2.5.1** Various arithmetic operators and explain the evaluation of arithmetic expressions with example.
 - 2.5.2** Various relational operators and discuss evaluation of relational expressions
 - 2.5.3** Various logical operators and discuss evaluation of logical expressions
- 2.6** Explain the difference between unary and binary operators
- 2.7** Describe various assignment operators, increment and decrement operators
- 2.8** Illustrate nested assignment
- 2.9** Explain conditional operators with an example
- 2.10** Explain
 - 2.10.1** Bit-wise operators and explain each with an example
 - 2.10.2** Special operators with examples
 - 2.10.3** Precedence and Associativity of operators
- 2.11** Describe evaluation of compound expression
- 2.12** Illustrate type conversion techniques
- 2.13** Write sample programs by using all the operators

- 3.0** Decision making, iterative and other control statements
- 3.1** Explain decision making statements and its need in programming
- 3.2** Explain
 - 3.2.1** Simple if and if-else statement with syntax and sample program
 - 3.2.2** Nested if..else statements with syntax and sample program
 - 3.2.3** if-else-if ladder with syntax and sample program
 - 3.2.4** switch statement with syntax and sample program
- 3.3** State the importance of break statement with switch and illustrate
- 3.4** Compare
 - 3.4.1** Conditional operator with if-else statement
 - 3.4.2** if-else with switch statement
- 3.5** Define looping or iteration
- 3.6** List and explain iterative statements with syntax and examples
- 3.7** Compare different loop statements
- 3.8** What is nested loop and illustrate.
- 3.9** Explain the usage of goto, break and continue statements with loop statements
- 3.10** Differentiate break and continue statements.
- 3.11** Define structured programming.
- 4.0** Arrays, strings, Structures and Unions
- 4.1** Define Array
- 4.2** Describe
 - 4.2.1** Declaration and initialization of One Dimensional(1D) Array with syntax and sample programs.
 - 4.2.2** Accessing the elements in 1D-Array with sample programs.
 - 4.2.3** Reordering an array in ascending order.
- 4.3** Explain declaration and initialization and usage of two Dimensional(2D)Arrays.
- 4.4** Illustrate the concept of arrays with sample programs on matrix addition, subtraction and matrix multiplication
- 4.5** Define String
- 4.6** Describe
 - 4.6.1** Declare and initialize of String variables.
 - 4.6.2** gets() and puts()
 - 4.6.3** Reading and displaying of strings from terminal with sample programs.

- 4.6.4 Explain about various String handling functions with sample programs.
- 4.7 Explain Character arithmetic.
- 4.8 Define a structure.
- 4.9 Explain
 - 4.9.1 Initializing structure, Declaring structure, Declaring Structure Variables.
 - 4.9.2 Accessing of the structure members
 - 4.9.3 Structure assignment.
 - 4.9.4 How to find size of a structure.
 - 4.9.5 Nested structure concept.
 - 4.9.6 Structures containing arrays
 - 4.9.7 Array of structures
- 4.10 Define Union, declare, initialize and use of union.
- 4.11 Distinguish between Structures and Unions
- 4.12 Write sample programs for all the concepts of structures and unions
- 5.0 User defined functions, pointers, file management and preprocessor directives
- 5.1 Explain
 - 1. Need of user defined functions
 - 2. Advantages of the functions
 - 3. Elements of function
 - 4. Return values and their types
- 5.2 Define a function call, function prototype
- 5.3 Explain
 - 5.3.1 Function declaration in programs
 - 5.3.2 Functions with no arguments and no return values with sample programs
 - 5.3.3 Functions with arguments with no return values with sample programs
 - 5.3.4 Functions with arguments with return values with sample programs
 - 5.3.5 Functions with no arguments with return values with sample programs
 - 5.3.6 Functions that return multiple values with sample programs
 - 5.3.7 Recursion with sample programs
 - 5.3.8 Passing arrays to functions with sample programs
 - 5.3.9 Structure as function arguments and structures as function values.
 - 5.3.10 Structures containing pointers.
 - 5.3.11 Self referential structures with examples.
 - 5.3.12 Storage classes-auto, register, static, extern
 - 5.3.13 Scope, visibility and lifetime of variables in functions
- 5.4 Differentiate Local and External variables
- 5.5 Define Global variable
- 5.6 Discuss passing the global variables as parameters using sample programs
- 5.7 Explain
 - 5.7.1 Declaration and initialization of Pointers.
 - 5.7.2 Accessing the address of a variable using &operator
 - 5.7.3 Accessing the value of a variable through pointer
 - 5.7.4 Pointer Arithmetic
 - 5.7.5 Precedence of address and de-referencing operators.
 - 5.7.6 Relationship between arrays and pointers.
 - 5.7.7 Accessing array elements using pointers
 - 5.7.8 Pointers as function arguments
 - 5.7.9 Pointer arrays with examples.
- 5.8 Differentiate between address and de-referencing operators.

- 5.9 Explain
 - 5.9.1 Dynamic memory management functions with examples.
 - 5.9.2 Structures containing pointers.
 - 5.9.3 Pointer to structure.
 - 5.9.4 Self referential structures with examples.
- 5.10 Explain
 - 5.10.1 Files and how to declare file pointer to a file
 - 5.10.2 Illustrate the concept of file opening using various modes
 - 5.10.3 Illustrate the concept of closing of a file
 - 5.10.4 Illustrate the concept of Input / Output operations on a file
 - 5.10.5 Illustrate the concept of random accessing files
 - 5.10.6 Explain different file handling functions
- 5.11 Explain
 - 5.11.1 Preprocessor directives
 - 5.11.2 Need of preprocessor directives.
- 5.12 Write
 - 5.12.1 Simple programs using preprocessor directives.
 - 5.12.2 Simple program using command line arguments (argc and argv)

COURSE CONTENT

1. **Introduction to C Language:** History of C language - importance of C Define language - structure of C language - programming style of C language - steps involved in executing the C program-Character set - C Tokens - Keywords and Identifiers- Constants and Variables - Data Types and classification - declaration of constants and variables- initializing values to variables-user defined data types-usage of type qualifiers.
2. **Input and output statements, Operators and Expressions in C:** importance of Pre-processor #include-reading and writing a single character functions- formatted input and output statements-operators-classification of operators-operator precedence and associativity- expressions and expression evaluation-type conversion techniques.
3. **Understand Decision making, iterative and other control statements :**simple if,if-else, if else ladder, nested if-else-switch statement - else if, nested if , else if ladder, switch statements- Classification of various loop statements- while statement – do.. while statement ram - for loop statement - nesting of loops- Comparisons of different loop statements –go to statement-break and continue statements –concept of structured programming.
4. **Understand Arrays and strings , basics of Structures and Unions:** Arrays -One Dimensional Arrays – array programs -two Dimensional Arrays- programs on matrix - Strings — String handling functions - Structure- Array of structures - Nested structures- pointer to structure Self referential structures - Union and illustrate use of a union – difference between Structures and Union

5. Understand User defined functions, basics of pointers, file management and preprocessor directives: Function – user defined functions – Advantages - Recursion concept - parameter passing –storage classes - scope, visibility and lifetime of variables in functions- Local and External variables -Global variable- - Pointer - Differentiate address and de-referencing operators - Pointer Arithmetic- precedence of address and de-referencing operators - -Relationship between Arrays and Pointers - Pointers as Function Arguments - Dynamic memory management- Files - file pointers - file opening in various modes - Concept of closing of a file –operations on files - Need of Preprocessor directives - Various Preprocessor directives- Macros – Command line arguments

REFERENCE BOOKS

Programming in ANSI C	E. Balaguruswamy	TataMcGrawHill
Programming with C	Gottfried	Tata McGrawHill
C The complete Reference	Schildt	Tata McGrawHill

Table specifying the scope of syllabus to be covered for unit tests

Unit Test	Learning outcomes to be covered
Unit test-1	From 1.1 to 2.13
Unit test-2	From 3.1 to 4.6
Unit test-3	From 4.7 to 5.12